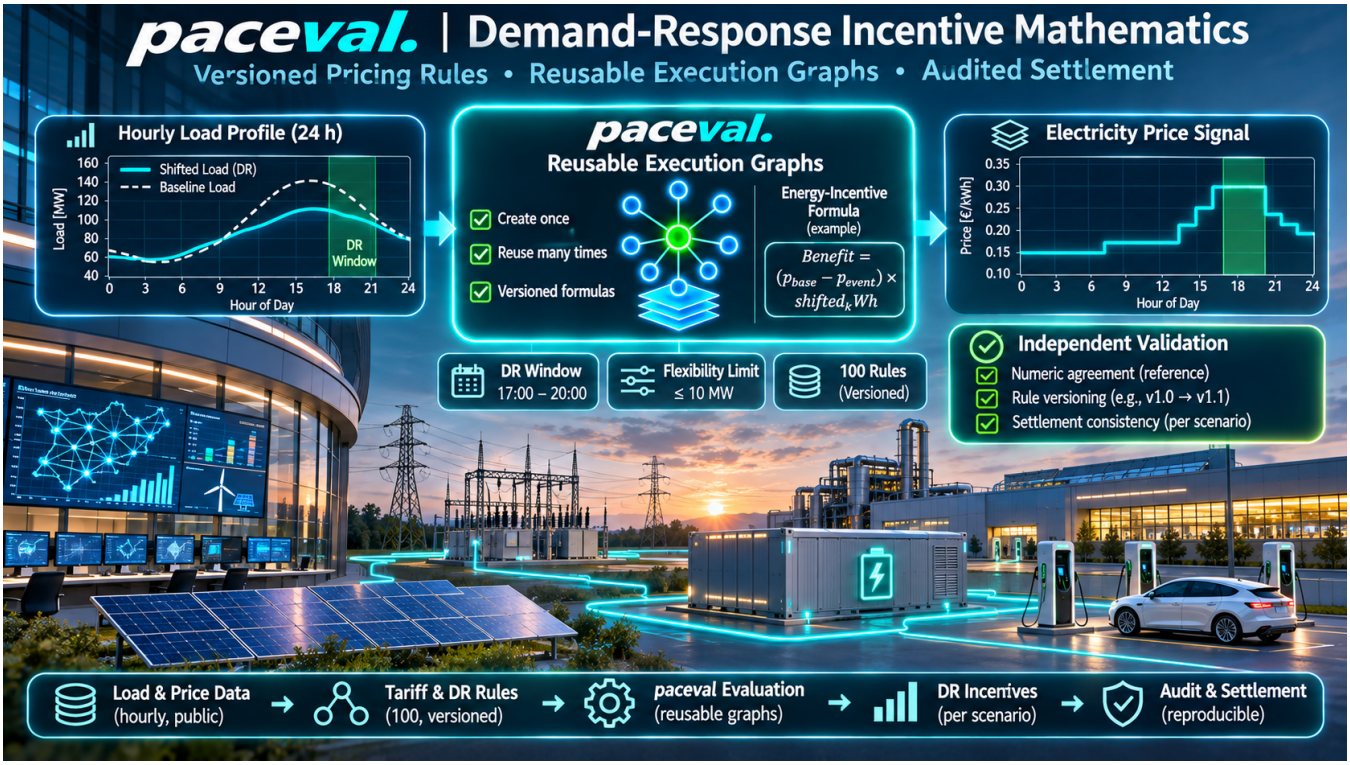
A REAL SDK IMPLEMENTATION • SYNTHETIC HOURLY LOAD • AUDITED MATHEMATICS

Demand-Response Incentive Mathematics

Versioned mathematical rules for flexible electricity demand - An executable C++ demonstration of reusable paceval computation graphs, measured-load eligibility, independent numerical verification and controlled formula updates.



Measured result: 100 reusable graphs; 336 one-hour records; 33,600 rule evaluations; 33,600 independently validated outputs; 24/24 controlled TINC samples.

This study converts the supplied WP-014 pilot proposal into a reproducible mathematical research demonstrator. Its physical time series, prices, consumption behavior and programs are deliberately synthetic. No financial settlement rule, external network constraint, field deployment or customer behavior has been validated.

1. Research question and system boundary

Can a library of one hundred independently configurable and versioned demand-response incentive expressions be compiled once, evaluated repeatedly for new hourly inputs and independently audited, while keeping measurement quality and physical constraints outside the arithmetic engine?

Why the use case is technically meaningful

Demand response changes electricity consumption in response to time-dependent prices or program incentives. The result depends on what the customer would otherwise have consumed (the counterfactual baseline), how much reduction was actually metered, eligibility rules and potential rebound. ACER and the U.S. Department of Energy distinguish measurement and verification from financial settlement; OpenADR provides a model for exchanging events and price signals. The paceval task is an explicit mathematical expression kernel, not a substitute for those upstream and downstream functions.

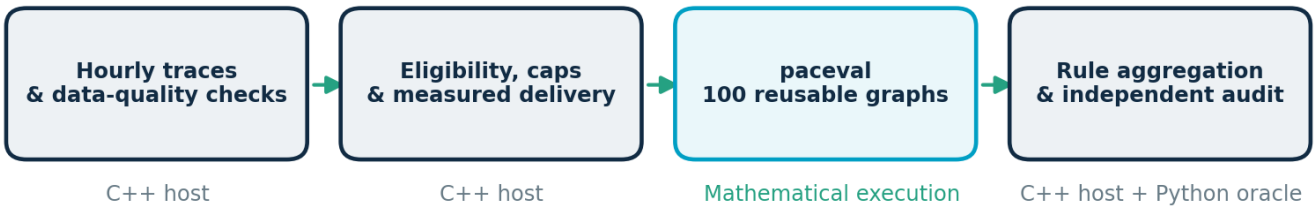


Figure 1. An intentionally modular implementation: the C++ host validates input and eligibility; paceval evaluates the 100 reusable mathematical rules; host and Python verify numerical outputs independently.

Extension beyond WP-014

The original proposal starts from a simple price-difference multiplied by shifted energy. The executable experiment adds: 24-hour energy recovery; fixed measured-load and capacity limits; controlled event-quality exclusions; four rule families and coefficient versioning; a native C++ oracle; a 90-digit decimal oracle; and a separate raw TINC diagnostic.

Boundary: a synthetic demand trace is not a verified customer baseline. Baseline estimation, actual consumer elasticity, distribution network feasibility and market-approved settlement remain independent research questions.

2. Mathematical formulation and units

One-hour values. Let B be the modeled baseline energy (kWh), A the synthetic meter energy (kWh), R the requested reduction (kWh), and C the declared capacity (kWh). For a quality-valid event the host computes d = min(R, C, max(0, B - A)). Outside an event or with bad meter quality, the host sets both d and the chargeable request to zero.

Explicit expression families evaluated by paceval

```
V_0 = alpha*d*(pe-pr) + bonus*d
V_1 = V_0 - penalty*(req-d)*(req-d)
V_2 = V_1 - comfort*d*d
V_3 = V_2 - cubic*d*d*d
```

These are four programmable research families, NOT prescribed retail, wholesale or balancing-market settlement formulas. Within the chosen model, the price-spread term represents a hypothetical benefit of moving consumption from an event hour to a later recovery hour; bonus is a separate synthetic program incentive; the remaining terms are specified under-delivery and comfort costs.

Quantity	Mathematical role / unit
d, req	Verified eligible / requested energy, kWh
pe, pr	Event and recovery reference prices, EUR/kWh
alpha	Dimensionless price-spread multiplier
bonus	Explicit program incentive assumption, EUR/kWh
penalty, comfort	Quadratic cost assumptions, EUR/kWh²
cubic	Cubic fatigue cost assumption, EUR/kWh³

Why the results are traceable

Each rule is stored with a stable ID, family and version, plus its exact C++-constructed paceval expression and literal coefficients. Thus a researcher can reconstruct the calculation from published `rules.csv` and `evaluations.csv` without inferring a hidden decision rule.

3. Controlled hourly energy experiment

Fourteen synthetic days yield 336 sequential one-hour samples. DR activation is fixed at hours 17, 18 and 19. All modeled energy reductions return during hours 20, 21 and 22, such that the sum of the baseline and metered loads agrees within a strict numerical tolerance on every day.

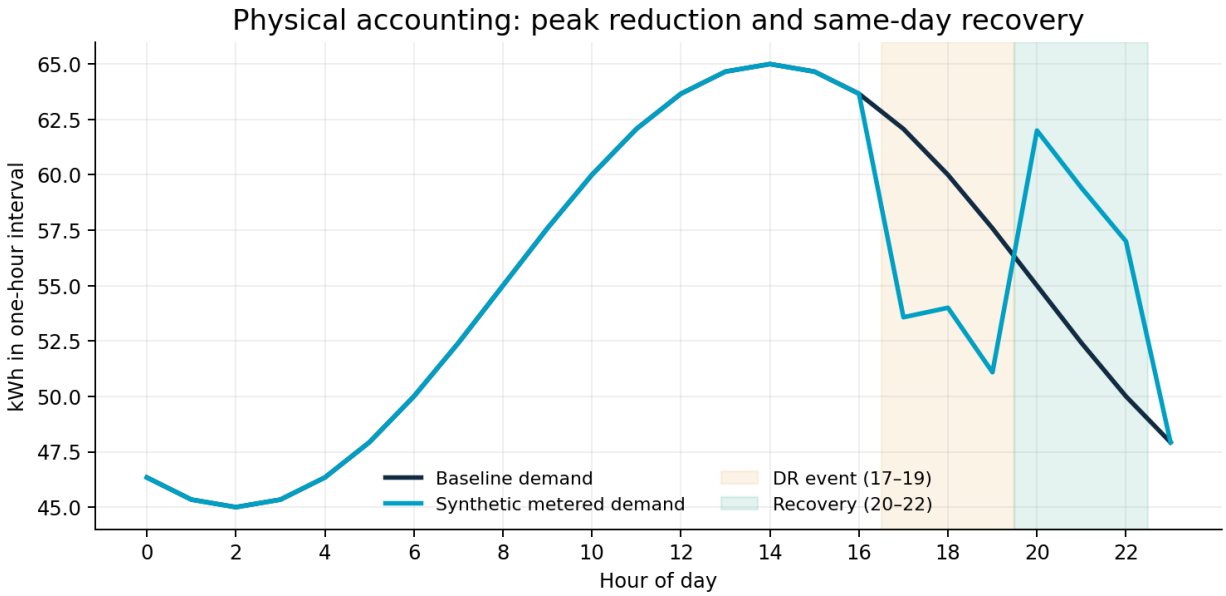


Figure 2. An example 24-hour profile. A reduction during the modeled peak is followed by an equal daily energy recovery; this avoids implicitly treating load shifting as energy conservation or deletion.

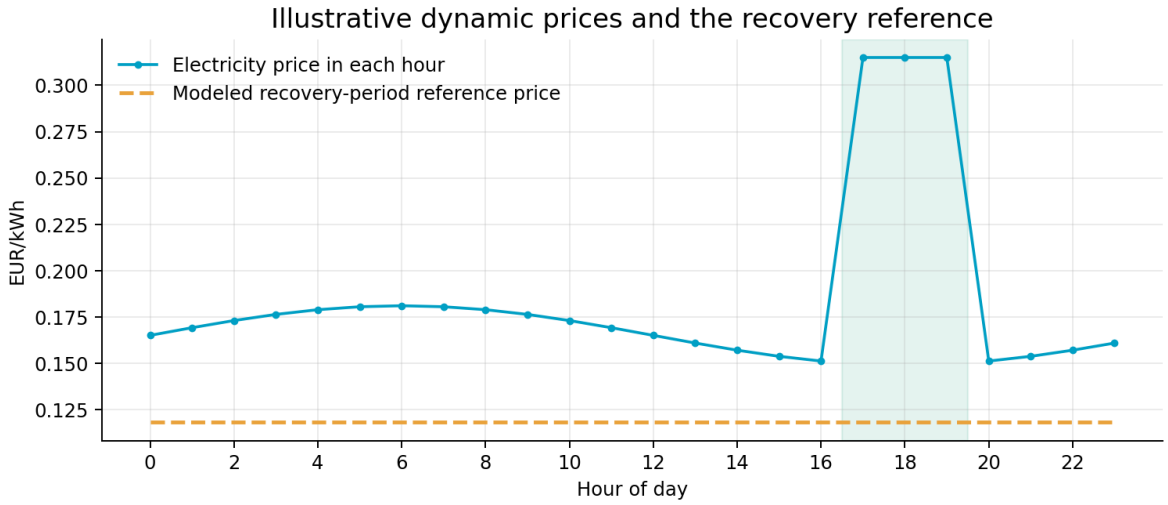


Figure 3. Illustrative price signals; no actual electricity market or supplier tariff is implied.

4. Direct paceval SDK integration

The executable compiles the original `paceval_main.cpp` facade next to `demand_response.cpp` and includes `paceval_main.h`. CMake links the user-supplied original Linux runtime as an external shared object. It deliberately does not use a custom `dlopen` API bridge, redistribute proprietary runtime binaries or compile its own mathematical engine.

Actual API roles

SDK call	Purpose in the executed C++ code
paceval_InitializeLibrary	Initializes the external paceval runtime
paceval_CreateComputation	Creates an independently reusable graph for each rule
paceval_dGetComputationResultExt	Batch-evaluates one graph for many hourly values
paceval_dGetComputationResult	Queries the separate TINC diagnostic graph
paceval_GetIsError	Checks computational errors
paceval_DeleteComputation	Releases each graph via C++ RAII
paceval_FreeLibrary	Closes the runtime after graph release

Reproducible evaluation lifecycle

The program creates one hundred graphs exactly once in the normal run. For every graph it fills a flattened batch with four numbers per hour (delivered, requested, event price and recovery price), invokes the official `...ResultExt` API, checks its per-sample errors, and compares results to a native C++ oracle. The SDK batch call reports a boolean true on error; this convention is checked explicitly in the source.

For a version change, rule 17 is recompiled with a modified bonus coefficient while already constructed rule 18 is re-evaluated unchanged. One graph changing does not invalidate the other 99.

Reproducibility is numerical and architectural. No runtime throughput, power consumption, monetary gain from actual demand response or FPGA speedup is claimed.

5. Numerical accuracy and independent audit

All 33,600 values produced by the original paceval Linux runtime pass the native C++ tolerance check. A separate Python audit uses 90-digit Decimal arithmetic and reconstructs the expressions from exported model coefficients and the actual IEEE-754 binary64 values supplied to the engine.

Verification	Result
Regular paceval vs native C++	33,600 / 33,600 pass
Independent 90-digit Decimal audit	33,600 / 33,600 pass
Maximum absolute discrepancy	approximately 6.943 × 10^-16 EUR
Raw TINC controlled samples	24 / 24 contain the reference
Daily energy-balance checks	14 / 14 pass

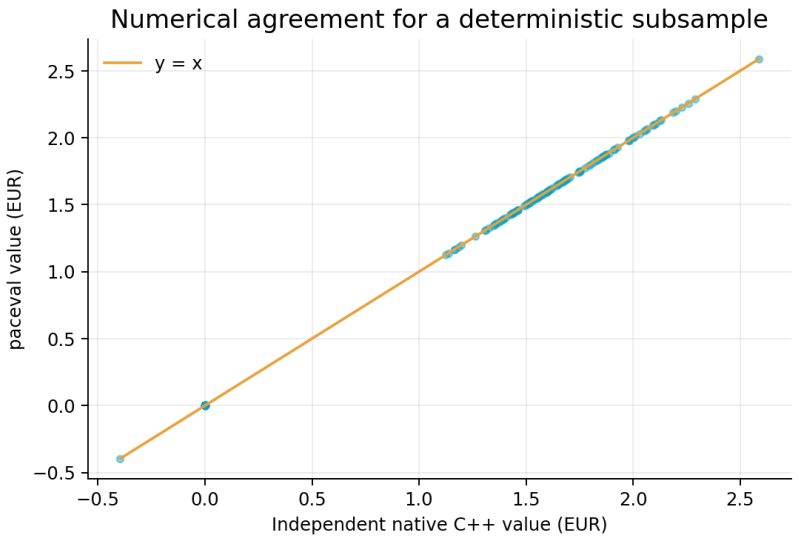


Figure 4. Paceval and native C++ reference agree across the deterministic subsample. The full audit is conducted on all exported rows, not only those displayed.

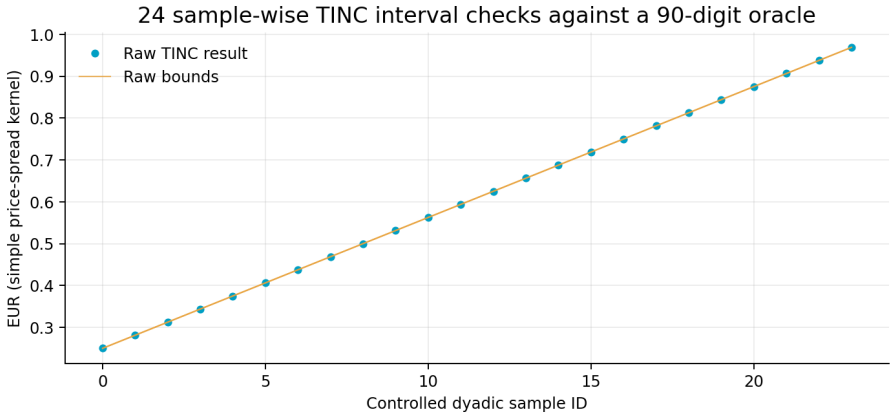


Figure 5. TINC uses a separate smooth price-spread expression on 24 binary-exact inputs. This is a sample-wise inclusion result, not a formal guarantee across arbitrary formulas.

6. Eligibility and false-incentive checks

The model demonstrates three important sources of misinterpretation: (i) a requested peak reduction without a metered reduction; (ii) a meter-quality-invalid hour; and (iii) an instruction greater than the declared flexible capacity. In the host, a claim is never accepted solely because it was requested; positive modeled payout is bounded by measured and capped delivery.

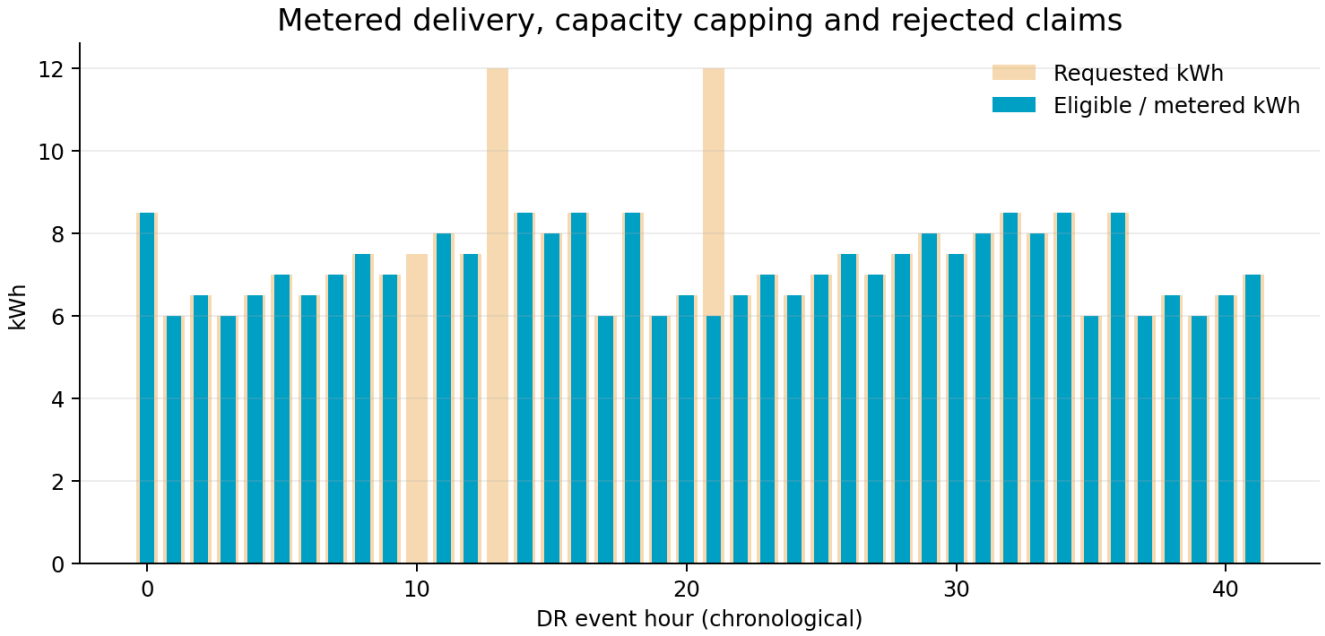


Figure 6. The requested quantity may exceed the independently eligible quantity; an invalid measurement is excluded instead of being rewarded.

Control	Observed result
Unsubstantiated requested reduction	1 deliberately injected event; no positive payout
Bad meter-quality event	1 excluded event hour
Requested amount above declared cap	2 capped event hours
24-hour physical reconciliation	Every synthetic day reconciles
Strict invalid CSV	Rejected with non-zero program exit

These checks prove the implemented host policy, not the accuracy or integrity of real customer baselines. Real DR settlement requires an appropriate program-specific baseline calculation and independent meter verification.

7. Formula library, revision and sensitivity

The rule generator produces one hundred distinct formula instances from four family structures with varying coefficients. The model library exposes a genuine engineering benefit of reusable expression objects: mathematical rules can be changed as versioned data without changing the host-side control flow.

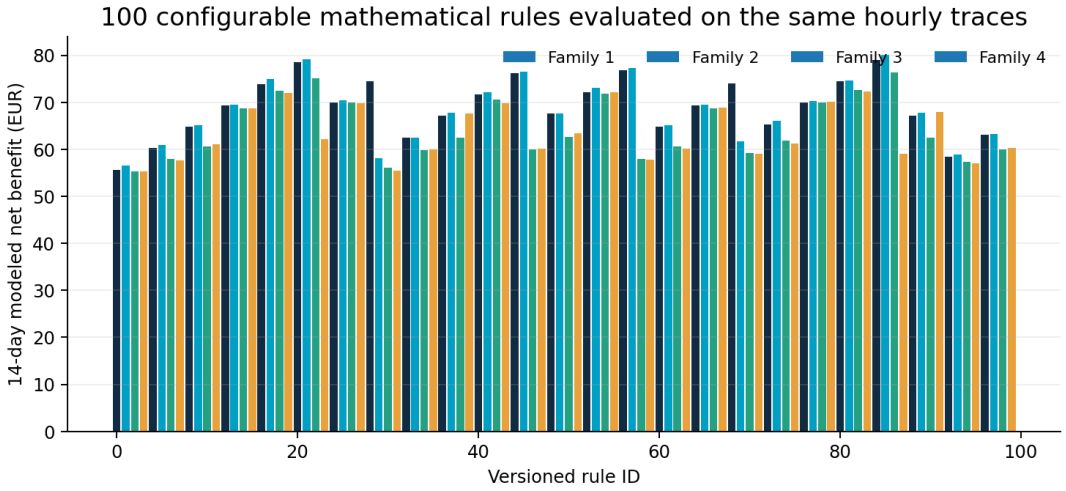


Figure 7. Synthetic modeled net benefits across one hundred independently constructed rule graphs. These are arithmetic scenario values, not real customer savings.

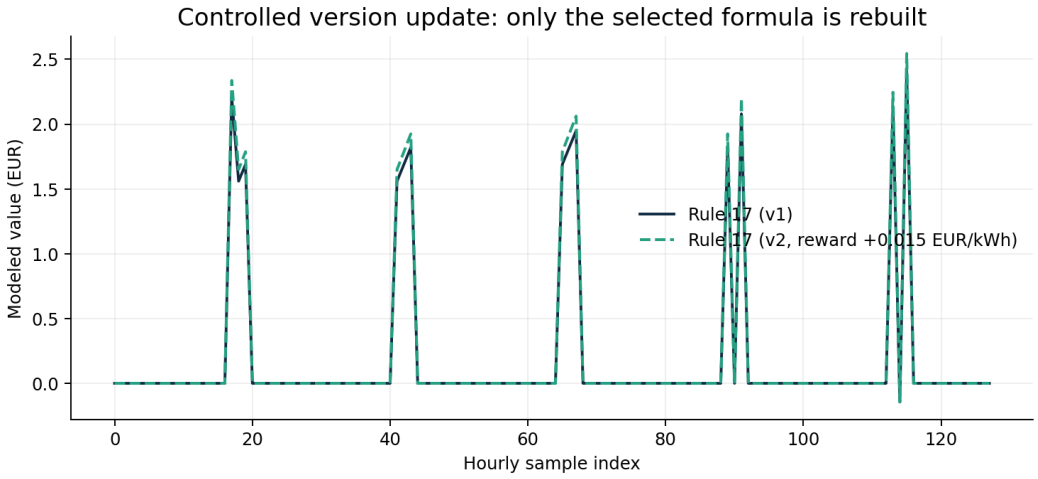


Figure 8. Controlled rule-17 update modifies the bonus coefficient by 0.015 EUR/kWh. The independent rule-18 computation remains unchanged.

The figures `05_rule_family_distribution` and `07_requested_vs_eligible_delivery` are also included in the accompanying GitHub repository as both PNG and SVG. All figures are produced by `src/make_figures.py` from the published CSV data.

8. Reproducibility, sources and next experiment

The archive is intentionally minimal: README, this PDF, the direct C++ SDK facade, CMake, validation scripts, the complete synthetic input/output CSVs and the marketing/scientific graphics. There is no `LICENSE_NOTICE.md` and no copy of the external Linux `.so`. Existing copyright/license notices in the official SDK source are preserved.

Run the complete experiment

```
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release
  -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so
cmake --build build -j
ctest --test-dir build --output-on-failure
./build/demand_response --rules 100 --days 14 --out reproduced
python3 src/validate_outputs.py --dir reproduced
```

References grounding the application specification

1. ACER (2025): Unlocking flexibility: Demand response barriers, including activation and aggregation context.

2. ACER (2025): Demand Response Network Code recommendation, including principles for baselines.

3. U.S. DOE (2013): Measurement and Verification for Demand Response, establishing separate measurement and settlement considerations.

4. OpenADR Alliance: OpenADR 3, price and event signal interoperability; no OpenADR live adapter was tested.

5. European Commission JRC (2022): Demand-response baseline impact evaluation review.

Next scientific step

Obtain permission-cleared hourly meter traces and corresponding officially defined DR program rules; compare candidate baseline methodologies on a pre-registered hold-out period; quantify settlement discrepancies and false-positive incentives against verified data; and extend the host with documented OpenADR input adapters and independent physical network constraints. Only those additional experiments could support claims beyond the current mathematical feasibility result.